

ФГАОУ ВО «НИУ ИТМО»
ФАКУЛЬТЕТ ПРОГРАММНОЙ ИНЖЕНЕРИИ
И КОМПЬЮТЕРНОЙ ТЕХНИКИ

Отчёт №3

Структуры данных

(по дисциплине «Алгоритмы и структуры данных»)

Выполнил:

Лабин Макар Андреевич,
АиСД 3.2, Р3231, 466449.

Проверил:

Смирнов Виктор Игоревич

ИТМО

г. Санкт-Петербург, Россия
2026

Содержание

1	Яндекс.Контест	1
1.1	Машинки	1
1.1.1	Описание решения	1
1.1.2	Асимптотическая сложность	1
1.1.3	Листинг кода	1
1.2	Гоблины и очереди	3
1.2.1	Описание решения	3
1.2.2	Асимптотическая сложность	3
1.2.3	Листинг кода	3
1.3	Менеджер памяти-1	4
1.3.1	Описание решения	4
1.3.2	Асимптотическая сложность	4
1.3.3	Листинг кода	4
1.4	Минимум на отрезке	7
1.4.1	Описание решения	7
1.4.2	Асимптотическая сложность	7
1.4.3	Листинг кода	7
2	Timus	9
2.1	War Games 2	9
2.1.1	Описание решения	9
2.1.2	Асимптотическая сложность	9
2.1.3	Листинг кода	9
2.2	White Streaks	10
2.2.1	Описание решения	10
2.2.2	Асимптотическая сложность	10
2.2.3	Листинг кода	10

1 Яндекс.Контекст

1.1 Машины

1.1.1 Описание решения

...

1.1.2 Асимптотическая сложность

Временная сложность решения — $\dots\mathcal{O}(\dots)$.

Пространственная сложность решения — $\dots\mathcal{O}(\dots)$.

1.1.3 Листинг кода

```
1 #include <algorithm>
2 #include <cstdint>
3 #include <iostream>
4 #include <map>
5 #include <set>
6 #include <vector>
7
8 namespace {
9     std::size_t INFTY = 5 * 1e5;
10
11     std::vector<std::size_t> CalculatePriorities(const std::vector<
12         std::size_t>& array) {
13         std::vector<std::size_t> priorities(array.size(), INFTY);
14         std::map<std::size_t, std::size_t> toy_history = {};
15         for (std::size_t i = 0; i < array.size(); i++) {
16             const std::size_t toy_type = array[i];
17             if (toy_history.count(toy_type) != 0U) {
18                 priorities[toy_history[toy_type]] = i;
19             }
20             toy_history[toy_type] = i;
21         }
22         return priorities;
23     }
24
25     struct Toy {
26         std::size_t idx;
27         std::size_t type;
28         bool used;
29
30         bool operator==(const Toy& other) const {
31             return type == other.type;
32         }
33     };
34
35     struct CompareByPriority {
36         const std::vector<std::size_t>& priorities;
```

```

37     explicit CompareByPriority(const std::vector<std::size_t>& arr
38         ) : priorities(arr) {
39     }
40     bool operator()(const Toy& l_toy, const Toy& r_toy) const {
41         if (priorities[l_toy.idx] != priorities[r_toy.idx]) {
42             return priorities[l_toy.idx] > priorities[r_toy.idx];
43         }
44         return l_toy.type < r_toy.type;
45     }
46 };
47 } // namespace
48
49 int main() {
50     std::size_t n_count = 0;
51     std::size_t k_count = 0;
52     std::size_t p_count = 0;
53
54     std::cin >> n_count >> k_count >> p_count;
55
56     std::vector<std::size_t> toys(p_count);
57     for (std::size_t i = 0; i < p_count; i++) {
58         std::cin >> toys[i];
59     }
60
61     std::size_t action_counter = 0;
62     const std::vector<std::size_t> priorities =
        CalculatePriorities(toys);
63
64     auto cmp = CompareByPriority(priorities);
65     std::set<Toy, decltype(cmp)> floor(cmp);
66     std::vector<Toy> active_toys(n_count + 1);
67
68     for (std::size_t i = 0; i < p_count; i++) {
69         const Toy current_toy = {i, toys[i], true};
70         if (active_toys[toys[i]].used) {
71             floor.erase(active_toys[toys[i]]);
72             floor.insert(current_toy);
73             active_toys[toys[i]] = current_toy;
74             continue;
75         }
76         if (floor.size() == k_count) {
77             const Toy top = *floor.begin();
78             floor.erase(top);
79             active_toys[top.type].used = false;
80         }
81         floor.insert(current_toy);
82         active_toys[toys[i]] = current_toy;
83         action_counter++;
84     }
85     std::cout << action_counter;

```

```
86     return 0;
87 }
```

1.2 Гоблины и очереди

1.2.1 Описание решения

...

1.2.2 Асимптотическая сложность

Временная сложность решения — $\dots O(\dots)$.

Пространственная сложность решения — $\dots O(\dots)$.

1.2.3 Листинг кода

```
1 #include <cstdint>
2 #include <iostream>
3 #include <list>
4
5 namespace {
6 void AddGoblin(
7     std::list<std::size_t>& first_part,
8     std::list<std::size_t>& second_part,
9     const std::size_t idx,
10    const bool is_privileged
11 ) {
12     if (first_part.empty() && second_part.empty()) {
13         first_part.push_back(idx);
14         return;
15     }
16     if (second_part.empty()) {
17         second_part.push_back(idx);
18         return;
19     }
20     if (is_privileged) {
21         if (first_part.size() == second_part.size()) {
22             first_part.push_back(idx);
23             return;
24         }
25         second_part.push_front(idx);
26         return;
27     }
28     second_part.push_back(idx);
29     if (first_part.size() < second_part.size()) {
30         const std::size_t middle_idx = second_part.front();
31         second_part.pop_front();
32         first_part.push_back(middle_idx);
33         return;
34     }
35 }
```

```

36
37 void PopGoblin(std::list<std::size_t>& first_part, std::list<std
    ::size_t>& second_part) {
38     const std::size_t to_pop = first_part.front();
39     first_part.pop_front();
40     std::cout << to_pop << "\n";
41
42     if (first_part.size() < second_part.size()) {
43         const std::size_t middle_idx = second_part.front();
44         second_part.pop_front();
45         first_part.push_back(middle_idx);
46     }
47 }
48 } // namespace
49
50 int main() {
51     std::size_t n_count = 0;
52     std::cin >> n_count;
53
54     std::list<std::size_t> first_part = {};
55     std::list<std::size_t> second_part = {};
56
57     for (std::size_t i = 0; i < n_count; i++) {
58         char action_type = '-';
59         std::cin >> action_type;
60         if (action_type == '-') {
61             PopGoblin(first_part, second_part);
62             continue;
63         }
64         std::size_t idx = 0;
65         std::cin >> idx;
66         AddGoblin(first_part, second_part, idx, action_type == '*');
67     }
68     return 0;
69 }

```

1.3 Менеджер памяти-1

1.3.1 Описание решения

...

1.3.2 Асимптотическая сложность

Временная сложность решения — $\dots\mathcal{O}(\dots)$.

Пространственная сложность решения — $\dots\mathcal{O}(\dots)$.

1.3.3 Листинг кода

```

1 #include <cstdint>
2 #include <cstdint>

```

```

3 #include <cstdlib>
4 #include <iostream>
5 #include <iterator>
6 #include <map>
7 #include <set>
8 #include <unordered_map>
9
10 namespace {
11 struct Block {
12     std::size_t start;
13     std::size_t size;
14
15     bool operator<(const Block& other) const {
16         if (size != other.size) {
17             return size < other.size;
18         }
19         return start < other.start;
20     }
21 };
22
23 void Allocate(
24     std::set<Block>& free_heap,
25     std::map<std::size_t, std::size_t>& free_blocks,
26     std::unordered_map<std::size_t, Block>& query_map,
27     const std::size_t alloc_size,
28     const std::size_t query_id
29 ) {
30     auto iter = free_heap.lower_bound({0, alloc_size});
31     if (iter != free_heap.end()) {
32         const Block block = *iter;
33         free_heap.erase(iter);
34         free_blocks.erase(block.start);
35
36         const std::size_t remaining_size = block.size - alloc_size;
37         if (remaining_size > 0) {
38             const std::size_t new_start = block.start + alloc_size;
39             free_heap.insert({new_start, remaining_size});
40             free_blocks[new_start] = remaining_size;
41         }
42         query_map[query_id] = {block.start, alloc_size};
43         std::cout << block.start + 1 << "\n";
44     } else {
45         std::cout << 1 << "\n";
46     }
47 }
48
49 void Deallocate(
50     std::set<Block>& free_heap,
51     std::map<std::size_t, std::size_t>& free_blocks,
52     std::unordered_map<std::size_t, Block>& query_map,
53     const std::size_t query_id

```

```

54 ) {
55     auto dict_it = query_map.find(query_id);
56     if (dict_it != query_map.end()) {
57         const Block allocated_block = dict_it->second;
58         query_map.erase(dict_it);
59
60         auto insert_res = free_blocks.insert({allocated_block.start,
61             allocated_block.size});
62         auto curr_it = insert_res.first;
63
64         auto next_it = std::next(curr_it);
65         if (next_it != free_blocks.end() &&
66             allocated_block.start + allocated_block.size == next_it
67             ->first) {
68             free_heap.erase({next_it->first, next_it->second});
69
70             curr_it->second += next_it->second;
71             free_blocks.erase(next_it);
72         }
73
74         if (curr_it != free_blocks.begin()) {
75             auto prev_it = std::prev(curr_it);
76             if (prev_it->first + prev_it->second == curr_it->first) {
77                 free_heap.erase({prev_it->first, prev_it->second});
78
79                 prev_it->second += curr_it->second;
80                 free_blocks.erase(curr_it);
81
82                 curr_it = prev_it;
83             }
84         }
85         free_heap.insert({curr_it->first, curr_it->second});
86     }
87 } // namespace
88
89 int main() {
90     std::size_t n_count = 0;
91     std::size_t m_count = 0;
92     std::cin >> n_count >> m_count;
93
94     std::set<Block> free_heap;
95     std::map<std::size_t, std::size_t> free_blocks;
96     std::unordered_map<std::size_t, Block> query_map;
97     free_heap.insert({0, n_count});
98     free_blocks[0] = n_count;
99
100     for (std::size_t i = 0; i < m_count; i++) {
101         int32_t item = 0;
102         std::cin >> item;
103         if (item > 0) {

```



```

103     const auto alloc_size = static_cast<std::size_t>(item);
104     Allocate(free_heap, free_blocks, query_map, alloc_size, i
        + 1);
105     continue;
106 }
107 const std::size_t query_id = std::abs(item);
108 Deallocate(free_heap, free_blocks, query_map, query_id);
109 }
110 return 0;
111 }

```

1.4 Минимум на отрезке

1.4.1 Описание решения

...

1.4.2 Асимптотическая сложность

Временная сложность решения — $\dots O(\dots)$.

Пространственная сложность решения — $\dots O(\dots)$.

1.4.3 Листинг кода

```

1 #include <cstdint>
2 #include <cstdlib>
3 #include <iostream>
4 #include <list>
5
6 struct EnumeratedItem {
7     std::size_t idx;
8     int32_t value;
9 };
10
11 namespace {
12 void DoStep(
13     std::list<EnumeratedItem>& window, const std::size_t
        curr_idx, const std::size_t window_size
14 ) {
15     int32_t item = 0;
16     std::cin >> item;
17     while (!window.empty() && window.back().value >= item) {
18         window.pop_back();
19     }
20     window.push_back({curr_idx, item});
21     if (curr_idx - window.front().idx + 1 > window_size) {
22         window.pop_front();
23     }
24 }
25 } // namespace
26

```

```

27 int main() {
28     std::size_t n_count = 0;
29     std::size_t k_count = 0;
30     std::cin >> n_count >> k_count;
31     if (k_count > n_count) {
32         std::cerr << "n_count must be greater than or equal to
           k_count";
33         return 1;
34     }
35
36     std::list<EnumeratedItem> window = {};
37     for (std::size_t i = 0; i < k_count; i++) {
38         DoStep(window, i, k_count);
39     }
40     std::cout << window.front().value << " ";
41     for (std::size_t i = k_count; i < n_count; i++) {
42         DoStep(window, i, k_count);
43         std::cout << window.front().value << " ";
44     }
45     return 0;
46 }

```

2 Timus

2.1 War Games 2

2.1.1 Описание решения

...

2.1.2 Асимптотическая сложность

Временная сложность решения — $\dots\mathcal{O}(\dots)$.

Пространственная сложность решения — $\dots\mathcal{O}(\dots)$.

2.1.3 Листинг кода

```
1 #include <cmath>
2 #include <cstdint>
3 #include <iostream>
4 #include <vector>
5
6 namespace {
7 struct SqrtBlock {
8     std::size_t sum;
9     std::vector<std::size_t> soldiers;
10 };
11
12 std::vector<SqrtBlock> InitCircle(std::size_t n_count) {
13     auto n_sqrt_c = static_cast<std::size_t>(std::ceil(std::sqrt(
14         n_count)));
15     std::vector<SqrtBlock> circle(n_sqrt_c, {0, {}});
16
17     std::size_t soldier_idx = 1;
18     for (std::size_t i = 0; i < n_sqrt_c; i++) {
19         while (soldier_idx <= n_count && circle[i].sum < n_sqrt_c) {
20             circle[i].soldiers.push_back(soldier_idx);
21             circle[i].sum++;
22             soldier_idx++;
23         }
24     }
25     return circle;
26 }
27
28 std::size_t FindAndRemove(std::vector<SqrtBlock>& circle, std::
29     size_t pos) {
30     for (auto& block : circle) {
31         if (pos <= block.sum) {
32             const auto offset = static_cast<std::ptrdiff_t>(pos - 1);
33             const auto iter = block.soldiers.begin() + offset;
34             const std::size_t val = *iter;
35             block.soldiers.erase(iter);
36             block.sum--;
37             return val;
38         }
39     }
40 }
```

```

36     }
37     pos -= block.sum;
38 }
39 return 0;
40 }
41
42 void ProcessCircle(std::vector<SqrtBlock>& circle, std::size_t
    n_count, std::size_t k_count) {
43     std::size_t current_pos = k_count;
44     for (std::size_t m_count = n_count; m_count >= 1; m_count--) {
45         const std::size_t soldier_id = FindAndRemove(circle,
            current_pos);
46         std::cout << soldier_id << " ";
47
48         if (m_count > 1) {
49             current_pos = (current_pos + k_count - 1) % (m_count - 1);
50             if (current_pos == 0) {
51                 current_pos = (m_count - 1);
52             }
53         }
54     }
55 }
56 } // namespace
57
58 int main() {
59     std::size_t n_count = 0;
60     std::size_t k_count = 0;
61     std::cin >> n_count >> k_count;
62
63     std::vector<SqrtBlock> circle = InitCircle(n_count);
64     ProcessCircle(circle, n_count, k_count);
65     return 0;
66 }

```

2.2 White Streaks

2.2.1 Описание решения

...

2.2.2 Асимптотическая сложность

Временная сложность решения — $\dots\mathcal{O}(\dots)$.

Пространственная сложность решения — $\dots\mathcal{O}(\dots)$.

2.2.3 Листинг кода

```

1 #include <algorithm>
2 #include <cstdint>
3 #include <iostream>
4 #include <set>

```

```

5 #include <vector>
6
7 namespace {
8 struct Point {
9     std::size_t x;
10    std::size_t y;
11
12    bool operator<(const Point& other) const {
13        if (x != other.x) {
14            return x < other.x;
15        }
16        return y < other.y;
17    }
18 };
19
20 void AddBounds(std::vector<Point>& arr, const std::size_t
    x_bound, const std::size_t y_bound) {
21     for (std::size_t y_idx = 0; y_idx <= y_bound + 1; y_idx++) {
22         for (const std::size_t x_idx : {static_cast<std::size_t>(0),
            x_bound + 1}) {
23             arr.push_back({x_idx, y_idx});
24         }
25     }
26     for (std::size_t x_idx = 1; x_idx < x_bound + 1; x_idx++) {
27         for (const std::size_t y_idx : {static_cast<std::size_t>(0),
            y_bound + 1}) {
28             arr.push_back({x_idx, y_idx});
29         }
30     }
31 }
32
33 bool SortByY(const Point& lpt, const Point& rpt) {
34     if (lpt.y != rpt.y) {
35         return lpt.y < rpt.y;
36     }
37     return lpt.x < rpt.x;
38 }
39 } // namespace
40
41 int main() {
42     std::size_t m_count = 0;
43     std::size_t n_count = 0;
44     std::size_t k_count = 0;
45     std::cin >> m_count >> n_count >> k_count;
46
47     std::vector<Point> black_points(k_count);
48     black_points.reserve(k_count + 2 * (n_count + m_count + 2));
49     for (std::size_t i = 0; i < k_count; i++) {
50         std::cin >> black_points[i].x >> black_points[i].y;
51     }
52     AddBounds(black_points, m_count, n_count);

```

```

53
54 std::size_t white_streaks = 0;
55 std::set<Point> suspicious_points = {};
56 std::sort(black_points.begin(), black_points.end());
57 for (std::size_t i = 0; i < black_points.size() - 1; i++) {
58     const Point& lpt = black_points[i];
59     const Point& rpt = black_points[i + 1];
60     if (lpt.x != rpt.x) {
61         continue;
62     }
63     const auto diff = static_cast<std::ptrdiff_t>(rpt.y - lpt.y)
64         ;
65     if (diff == 1) {
66         continue;
67     }
68     if (diff == 2) {
69         suspicious_points.insert({lpt.x, lpt.y + 1});
70         continue;
71     }
72     white_streaks++;
73 }
74
75 std::sort(black_points.begin(), black_points.end(), SortByY);
76 for (std::size_t i = 0; i < black_points.size() - 1; i++) {
77     const Point& lpt = black_points[i];
78     const Point& rpt = black_points[i + 1];
79     if (lpt.y != rpt.y) {
80         continue;
81     }
82     const auto diff = static_cast<std::ptrdiff_t>(rpt.x - lpt.x)
83         ;
84     if (diff == 1) {
85         continue;
86     }
87     if (diff == 2) {
88         const Point sus_point = {lpt.x + 1, lpt.y};
89         if (suspicious_points.count(sus_point) != 0U) {
90             suspicious_points.erase(sus_point);
91             white_streaks++;
92         }
93         continue;
94     }
95     white_streaks++;
96 }
97 std::cout << white_streaks;
98 return 0;
99 }

```